

Locking Pages in Memory

1 Core Idea

Definition

Locking a page in memory, also called **pinning**, means marking a page frame so the page replacement algorithm is not allowed to evict it.

Virtual memory does not remove the need for I/O. When I/O devices read or write data, they often operate directly on physical memory. This creates a problem: paging may try to move or reuse a page while an I/O device is still using it.

Main Rule

A page involved in an active I/O operation must stay in memory until that I/O operation finishes.

2 Where the Problem Comes From

Suppose a process issues a system call such as:

```
read(file, user_buffer, size)
```

The buffer belongs to the process' virtual address space. The operating system translates that buffer to one or more physical page frames so the device can place the incoming data there.

Step	What happens
1	Process starts an I/O request into a user buffer.
2	The process blocks while the device is working.
3	Another process runs and may cause a page fault.
4	A global page replacement algorithm chooses some page frame to evict.
5	If it chooses the buffer page, the active I/O transfer is corrupted.

3 DMA Hazard

DMA

DMA means Direct Memory Access. A device controller transfers data directly to or from physical memory without the CPU copying every byte.

DMA uses physical addresses. Once the I/O operation begins, the device expects the target physical frame to remain the same.

What Can Go Wrong

If the buffer page is evicted during DMA, the physical frame may be reused for another page. The device may then write part of the data into the intended buffer and the rest over an unrelated page.

The bug is especially dangerous because the process that gets damaged may not be the process that requested the I/O.

4 Why Global Replacement Makes This Possible

With a **local** replacement policy, a process normally loses only its own pages. With a **global** replacement policy, any process' page may be selected as the victim.

Policy	Risk during I/O
Local replacement	Lower risk, because the faulting process usually replaces one of its own pages.
Global replacement	Higher risk, because the victim may be a page that belongs to a blocked process doing I/O.

Global replacement is useful for memory utilization, but it needs extra protection for pages that must not move.

5 Solution 1: Pin the I/O Pages

The operating system can mark pages used in an active I/O operation as pinned.

Pinning Rule

pinned page $\Rightarrow$ not eligible for page replacement

The basic sequence is:

1. Translate the user buffer into physical page frames.
2. Pin those page frames.
3. Start the device I/O operation.
4. Keep the process blocked while I/O is in progress.
5. When the device interrupt reports completion, unpin the pages.

Effect

Pinning makes the page replacement algorithm skip those frames, so the device always writes to the correct physical memory.

6 Solution 2: Use Kernel Buffers

Another approach is to avoid device I/O directly into user pages.

Stage	Action
Device I/O	Device transfers data into a kernel buffer.
After I/O completes	OS copies the data from the kernel buffer to the user buffer.

Kernel buffers are controlled by the operating system, so the OS can keep them resident and avoid exposing user pages directly to DMA.

7 Pinning vs Kernel Buffers

Method	Advantage	Cost
Pin user pages	Avoids an extra copy; good for large transfers.	Pinned pages reduce the number of frames available for replacement.
Kernel buffer	Simpler control; avoids DMA directly into pageable user memory.	Requires copying data between kernel memory and user memory.

8 Design Notes

- Pin pages only for the duration of the I/O operation.
- Too many pinned pages can make memory pressure worse.
- Page replacement code must check the pinned status before selecting a victim.
- Kernel buffers trade CPU copy overhead for simpler and safer I/O handling.

9 Final Summary

One-Box Summary

I/O devices may use physical frames directly.
A page used by active I/O must not be evicted.
Pinning locks the page in memory until I/O completes.
Kernel buffers avoid direct I/O into pageable user pages.